

Trabajo Práctico 4 – Sistemas de Computación



Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

Link del repositorio: https://github.com/Javier-Sinuka/computer_systems_ramnt/tree/main/TP4

1	Desarrollo	4
1.1	Compilación y preparación del módulo	4
1.2	Verificación de carga del módulo	5
1.3	Descarga del módulo y nueva verificación	5
1.4	Inspección de metadatos con modinfo	6
2	Discusión y análisis complementario	9
2.1	1. ¿Qué diferencias se pueden observar entre los dos modinfo?	9
2.2		
2.	¿Qué drivers/módulos están cargados en sus propias PC? Comparar las salidas con las computadoras de cada integrante del grupo. Expliquen las diferencias.	10
2.2.1	2.2 Archivos de relevamiento y comparación	10
2.3		
3.	¿Cuáles no están cargados pero están disponibles? ¿Qué pasa cuando el driver de un dispositivo no está disponible?	10
2.4		
4.	Correr hwinfo en una PC real con hardware real y agregar la URL de la información de hardware en el reporte.	11
2.5	5. ¿Qué diferencia existe entre un módulo y un programa?	11
2.6		
6.	¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple helloworld en C?	12
2.7		
7.	¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y cómo lo hace un programa?	12
2.8		
8.	¿Se animan a intentar firmar un módulo de kernel? ¿Y documentar el proceso?	13
2.9		
10.	¿Qué pasa si mi compañero con Secure Boot habilitado intenta cargar un módulo firmado por mí?	19
2.10	11. Dada la siguiente nota	20
2.10.1		

2.10.2

b. ¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo? 20

2.10.3

c. ¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema? 20

3 Respuestas 21

3.1 Desafío 1 21

3.1.1 ¿Qué es checkinstall y para qué sirve? 21

3.2 Desafío 2 21

3.2.1

¿Qué funciones tiene disponibles un programa y qué funciones tiene disponibles un módulo? 21

3.2.2 Espacio de usuario y espacio del kernel 22

3.2.3 Espacio de datos 22

3.2.4 Drivers e investigación del contenido de /dev 22

3.3 Bibliografía 23

1 Desarrollo

En esta sección se documenta el procedimiento realizado para compilar, cargar, inspeccionar y descargar un módulo del kernel en Linux. La secuencia se expone de manera procedural, siguiendo el orden de ejecución observado en las capturas relevadas durante la práctica. El trabajo se realizó sobre el directorio correspondiente a la primera parte del proyecto de módulos, y posteriormente se emplearon herramientas de inspección del kernel para verificar tanto el estado de carga como los metadatos del módulo.

1.1 Compilación y preparación del módulo

En primer lugar, se ingresó al directorio de trabajo correspondiente a la primera parte del ejercicio y se ejecutó `make` sobre el módulo. Esta acción invocó el sistema de compilación del kernel para construir el objeto cargable `mimodulo.ko`. Tal como se verificó previamente, la compilación no presentó errores fatales: el proceso generó el artefacto esperado del módulo y dejó disponible el binario para su posterior inserción en el kernel.

Una vez finalizada la compilación, se procedió a cargar el módulo mediante `sudo insmod mimodulo.ko`. Dado que `insmod` no suele producir una salida descriptiva por sí mismo cuando la operación es exitosa, la validación de la carga se realizó consultando el buffer de mensajes del kernel con `sudo dmesg`.

```
1  cd part1/module
2  make
3  sudo insmod mimodulo.ko
4  sudo dmesg
```

La salida obtenida en `dmesg` evidenció tres aspectos relevantes: el kernel detectó la carga de un módulo externo (`out-of-tree`), informó que la verificación de firma no pudo completarse por ausencia de una clave requerida, y finalmente confirmó que el módulo fue cargado correctamente. Esto resulta coherente con una práctica académica en la que se trabaja con un módulo propio no firmado por una clave de confianza del sistema.

```

[70100.402025] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70100.402053] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70100.402061] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70100.402070] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70110.718729] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70110.718778] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70110.718790] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70110.718803] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70113.342984] mimodulo: loading out-of-tree module taints kernel.
[70113.342989] mimodulo: module verification failed: signature and/or required key missing - tainting ke
rnel
[70113.343536] Modulo cargado en el kernel.
(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa

```

1.1: Salida de dmesg luego de cargar mimodulo

1.2 Verificación de carga del módulo

Luego de la inserción, se utilizó `lsmod | grep mod` para verificar que el módulo efectivamente había quedado registrado entre los módulos actualmente cargados. La salida mostró la entrada correspondiente a `mimodulo`, junto con su tamaño en memoria y su contador de uso.

```
lsmod | grep mod
```

```

(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa
• rtl/module$ lsmod | grep mod
mimodulo                12288  0
(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa
○ rtl/module$

```

1.2: Verificación de mimodulo mediante lsmod

La presencia de `mimodulo` en esta lista confirmó que el kernel había aceptado la inserción del módulo y que este permanecía residente en memoria al momento de la consulta.

1.3 Descarga del módulo y nueva verificación

En una segunda etapa se procedió a retirar el módulo del kernel con `sudo rmmod mimodulo`. Una vez realizada la operación, se consultó nuevamente `dmesg` para observar el mensaje asociado a la descarga, y se volvió a ejecutar `lsmod | grep mod` para comprobar que el módulo ya no figuraba entre los módulos activos.

```

1  sudo rmmod mimodulo
2  sudo dmesg

```

La evidencia obtenida mostró el mensaje `Modulo descargado del kernel.` y, a continuación, la ausencia de `mimodulo` en la salida de `lsmod`, lo cual indica que la descarga se realizó de forma satisfactoria.

```
[70283.688808] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70283.688816] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70283.688825] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70298.577039] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70298.577069] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70298.577077] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70298.577086] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70307.093729] Modulo descargado del kernel.
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
rtl/module$ lsmod | grep mod
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
rtl/module$
```

1.3: Descarga del módulo y verificación posterior

Adicionalmente, se ejecutó el comando `cat /proc/modules | grep mod` desde la terminal. En este caso no se obtuvo ninguna salida. Este comportamiento es consistente con el estado del sistema luego de haber descargado el módulo, ya que `/proc/modules` enumera únicamente los módulos cargados en ese instante y el filtrado aplicado con `grep mod` no encontró coincidencias relevantes una vez removido `mimodulo`.

1.4 Inspección de metadatos con `modinfo`

Finalmente, se utilizaron consultas `modinfo` para inspeccionar los metadatos del módulo propio y compararlos con los de un módulo provisto por el sistema. En primer lugar, se consultó el archivo generado localmente:

```
modinfo mimodulo.ko
```

La salida permitió observar información característica del módulo compilado, incluyendo nombre, autor, descripción, licencia y la cadena `vermagic`, que expresa la compatibilidad del binario con la versión del kernel en ejecución.

1.4: Metadatos del módulo propio mediante modinfo

En segundo término, se inspeccionó un módulo perteneciente al árbol del sistema. Aunque en el esquema inicial la consulta se había pensado sobre una ruta terminada en `.ko`, en la práctica la ruta utilizada fue la que aparece en la captura, correspondiente al archivo comprimido del sistema:

```
modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
```

Esta consulta permitió advertir una diferencia importante respecto del módulo desarrollado en la práctica: el módulo del sistema presenta metadatos adicionales vinculados con la firma criptográfica, tales como identificador de firma, firmante, clave de firma y bloque de firma. Esto refleja el mecanismo de integridad empleado por el kernel para módulos distribuidos oficialmente por el sistema.

```
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
• rtl/module$ modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
filename:      /lib/modules/6.17.0-23-generic/kernel/crypto/des_generic.ko.zst
alias:         crypto-des3_edc-generic
alias:         des3_edc-generic
alias:         crypto-des3_edc
alias:         des3_edc
alias:         crypto-des-generic
alias:         des-generic
alias:         crypto-des
alias:         des
author:        Dag Arne Osvik <da@osvik.no>
description:   DES & Triple DES EDE Cipher Algorithms
license:       GPL
srcversion:    A4E91C81A384FB1FA6D530F
depends:        libdes
intree:        Y
name:          des_generic
retpoline:     Y
vermagic:      6.17.0-23-generic SMP preempt mod_unload modversions
sig_id:        PKCS#7
signer:        Build time autogenerated kernel key
sig_key:       64:D7:DF:5C:E3:FA:59:21:38:42:72:F2:91:87:8E:AA:03:09:1A:12
sig_hashalgo:  sha512
signature:     AA:55:3E:C4:12:C8:3D:FB:A7:39:70:FD:A8:3A:4B:B5:29:56:B6:FC:
12:C4:AC:69:B5:43:76:05:BD:20:9F:98:BC:4F:87:4D:5D:D9:0C:8F:
38:4A:33:FC:FC:84:3D:35:BF:7B:11:79:52:88:AC:72:3D:0C:50:29:
80:C6:0C:84:23:29:87:EB:4D:65:52:64:03:D0:35:FB:E3:C9:69:37:
92:B4:AE:2F:9D:1B:9B:49:D0:16:A4:99:DA:68:22:C4:47:C9:BE:FF:
66:59:91:4B:E0:38:3C:06:41:6E:56:00:F7:30:36:54:26:FD:06:05:
F7:D4:24:72:C8:00:F8:88:5F:0C:89:04:A0:E3:CD:5E:16:F7:73:28:
96:BC:CF:31:E6:B7:41:50:75:18:97:1D:AC:A9:92:A9:C3:17:32:D3:
56:1F:EE:EA:FA:EF:92:AB:09:81:5D:A0:80:3F:E0:80:49:88:B4:4E:
BD:40:7B:AB:62:2C:02:08:06:68:20:52:5E:A2:3C:F0:20:59:CD:14:
95:5C:A6:BA:DB:87:C9:9D:1D:7A:12:6C:41:E3:5F:50:18:91:C4:1B:
A9:D7:B8:D2:B9:6E:C9:3C:E0:F6:5A:22:0E:94:DF:06:AB:94:32:8B:
C8:CD:8C:2D:70:2B:EE:7F:BD:5B:5C:C8:31:B6:D3:9A:53:0B:2B:AC:
A8:5B:E0:60:29:25:86:25:2B:42:22:28:01:8D:40:8A:0D:78:83:D8:
B3:99:C7:3C:FB:BC:2D:30:58:74:20:1C:F3:02:5F:93:29:B1:4E:EE:
BE:46:0F:B4:53:CA:5A:86:34:7B:72:F4:F7:6B:F8:26:D6:2A:5C:73:
3C:00:42:95:67:22:30:BF:75:FA:92:BE:89:80:EB:C8:DE:6A:2C:45:
F1:B3:F9:29:DE:A8:39:C3:DC:38:83:91:9D:79:42:6C:36:AD:A0:70:
EE:74:DF:9A:C3:01:4B:18:14:F4:DC:18:59:8C:FF:04:C9:19:6F:D9:
38:42:8B:01:D3:A0:8A:DA:F4:73:08:B1:9E:F1:ED:19:45:CB:AD:F4:
5E:52:EF:A5:39:27:26:3C:08:58:79:BE:AC:60:07:23:95:82:24:6F:
1D:56:2F:AC:B9:49:32:52:B0:39:6C:79:62:78:17:64:56:EC:9B:B7:
AA:76:93:28:BC:71:1C:EE:56:A4:B7:58:EF:DD:B9:16:C3:11:46:56:
13:3E:B2:1A:E2:90:5B:B0:8A:5F:15:A2:12:2C:56:CB:EA:EA:04:DE:
C5:69:69:D2:0D:59:19:A7:04:9E:FC:B0:4A:E4:31:18:06:A2:17:8F:
98:58:97:62:47:89:51:BB:67:11:50:CC

(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
• rtl/module$
```

1.5: Inspección de un módulo del sistema con modinfo

2 Discusión y análisis complementario

La presente sección reúne una serie de preguntas de análisis vinculadas con la experiencia realizada sobre módulos del kernel, mecanismos de observación del sistema y consideraciones de seguridad asociadas a `Secure Boot`. Se responde únicamente aquello efectivamente trabajado durante la práctica, dejando como pendiente las consignas que todavía no fueron desarrolladas por el grupo.

2.1 1. ¿Qué diferencias se pueden observar entre los dos `modinfo`?

La primera diferencia observable es el origen de cada módulo. `mimodulo.ko` corresponde a un módulo desarrollado y compilado manualmente durante la práctica, mientras que `des_generic.ko.zst` forma parte del árbol de módulos provisto por el sistema operativo. Esta diferencia de procedencia se refleja directamente en los metadatos mostrados por `modinfo`.

En el caso de `mimodulo.ko`, la salida contiene información básica y mínima: nombre, autor, descripción, licencia, `srcversion`, `retpoline` y `vermagic`. Esto es consistente con un módulo académico simple, concebido para demostrar el proceso de compilación e inserción dinámica en el kernel. Por el contrario, el módulo del sistema exhibe una estructura de metadatos más rica: incluye múltiples alias, dependencias declaradas, el atributo `intree: Y`, e información explícita de firma criptográfica, como `sig_id`, `signer`, `sig_key`, `sig_hashalgo` y el bloque de `signature`.

Una segunda diferencia relevante reside en la confianza e integración con el kernel. El módulo del sistema está firmado y reconocido como parte del árbol oficial (`in-tree`), mientras que `mimodulo.ko` es un módulo `out-of-tree`, no firmado por una clave confiable para el sistema. Esto explica por qué, al cargarlo, el kernel emitió advertencias de verificación de firma y marcó la inserción como potencialmente contaminante (`tainting kernel`). En consecuencia, ambos `modinfo` no solo describen módulos distintos desde el punto de vista funcional, sino también desde el punto de vista de la política de integridad y confianza del kernel.

2.2 2. ¿Qué drivers/módulos están cargados en sus propias PC? Comparar las salidas con las computadoras de cada integrante del grupo. Expliquen las diferencias.

Consigna complementaria: cargar un archivo `.txt` con la salida de cada integrante en el repositorio y agregar un `diff` en el informe.

2.2.1 2.2 Archivos de relevamiento y comparación

Para sistematizar esta parte del trabajo se utilizó la carpeta `documentation/txt_drivers_modules`, donde se concentraron tanto los reportes generados por cada integrante como los scripts auxiliares empleados para producirlos y compararlos. Los archivos `agustin.txt`, `javier.txt` y `tomas.txt` contienen el relevamiento realizado sobre cada computadora, incluyendo dos bloques principales: los módulos cargados en el kernel y los drivers detectados a partir de `sysfs`.

La generación de estos reportes se automatizó mediante el script `drivers_modules`, cuyo propósito es inspeccionar la máquina local y exportar la información obtenida a un archivo de texto con formato uniforme. Sobre esa base, se desarrolló además el script `compare_drivers_modules`, que procesa todos los `.txt` presentes en la carpeta y produce por consola una comparación resumida entre ellos. El criterio adoptado no consiste en un `diff` textual bruto, sino en una comparación semántica: para módulos se considera únicamente el nombre del módulo cargado, mientras que para drivers se utiliza la combinación `bus/driver/módulo asociado`. De este modo, la salida permite distinguir con mayor claridad qué elementos son comunes entre los tres equipos y cuáles constituyen diferencias de hardware, controladores o configuración.

2.3 3. ¿Cuáles no están cargados pero están disponibles? ¿Qué pasa cuando el driver de un dispositivo no está disponible?

Un módulo puede estar disponible sin encontrarse cargado cuando existe físicamente en el árbol `/lib/modules/$(uname -r)` pero no aparece en la lista de módulos activos obtenida con `lsmod` o consultando `/proc/modules`. En otras palabras, la disponibilidad se refiere a que el sistema posee el archivo del módulo y puede, en principio, cargarlo si una

dependencia, un dispositivo o una acción administrativa lo requiere; la carga efectiva, en cambio, depende de que exista una necesidad concreta de activarlo.

Dentro de la práctica, el módulo `des_generic.ko.zst` constituye un ejemplo representativo de esta situación. Fue posible inspeccionarlo con `modinfo`, lo que demuestra que estaba instalado y disponible en el sistema, pero no surgió evidencia de que estuviera cargado en memoria al momento de las verificaciones realizadas. Esta distinción muestra que el conjunto de módulos instalados suele ser considerablemente más amplio que el subconjunto efectivamente cargado durante una sesión.

Cuando el driver de un dispositivo no está disponible, el kernel no puede asociar correctamente una implementación de control a ese hardware o a esa interfaz lógica. Como consecuencia, el dispositivo puede no ser detectado, puede aparecer identificado solo de manera genérica o puede quedar visible pero sin funcionalidad utilizable desde el espacio de usuario. En términos prácticos, esto implica pérdida total o parcial del servicio asociado: ausencia de conectividad en una placa de red, falta de audio, imposibilidad de montar un dispositivo de almacenamiento, o carencia de aceleración gráfica, entre otros casos.

2.4 4. Correr `hwinfo` en una PC real con hardware real y agregar la URL de la información de hardware en el reporte.

URL: <https://gist.github.com/Agustin-Tosolini/7c232021e65db62ec3a8f9801e77ae68>

2.5 5. ¿Qué diferencia existe entre un módulo y un programa?

La diferencia fundamental es que un programa convencional se ejecuta en espacio de usuario como un proceso aislado, con privilegios limitados y acceso al sistema a través de llamadas al sistema. Un módulo del kernel, en cambio, es una extensión dinámica del núcleo que se carga en espacio privilegiado y pasa a ejecutarse con los mismos privilegios que el kernel.

Desde el punto de vista operativo, un programa puede iniciarse, finalizar o fallar sin comprometer necesariamente al sistema completo, ya que el kernel lo aísla del resto de los procesos. Un módulo, por el contrario, comparte el contexto interno del kernel; por ello, un error en su lógica, en sus accesos a memoria o en su sincronización puede degradar la

estabilidad global del sistema e incluso provocar un bloqueo. En consecuencia, aunque ambos son piezas de software ejecutable, su nivel de privilegio, su modelo de interacción con el sistema y el impacto potencial de sus fallas son sustancialmente distintos.

2.6 6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple `helloworld` en C?

La forma más directa de observar las llamadas al sistema realizadas por un programa sencillo en C consiste en ejecutarlo mediante `strace`. Esta herramienta intercepta y muestra las `syscalls` invocadas por el proceso, así como sus argumentos y valores de retorno. Por ejemplo, luego de compilar un `helloworld`, puede ejecutarse `strace ./helloworld` para visualizar en pantalla llamadas tales como `execve`, `mmap`, `openat`, `write` y `exit_group`, entre otras.

Si se desea conservar la traza para su análisis posterior, resulta conveniente redirigir la salida a un archivo, por ejemplo mediante `strace -o traza.txt ./helloworld`. A su vez, si el interés principal es obtener un resumen estadístico de las llamadas efectuadas, puede utilizarse `strace -c ./helloworld`. Desde una perspectiva metodológica, `strace` permite vincular el comportamiento observable de un programa de espacio de usuario con la interfaz concreta mediante la cual ese programa solicita servicios al kernel.

2.7 7. ¿Qué es un `segmentation fault`? ¿Cómo lo maneja el kernel y cómo lo hace un programa?

Un `segmentation fault` es una excepción provocada por un acceso inválido a memoria por parte de un proceso, por ejemplo al intentar desreferenciar un puntero nulo, escribir en una página sin permisos de escritura o acceder a una dirección que no pertenece a su espacio virtual válido. En esencia, representa una violación de la política de protección de memoria impuesta por la arquitectura y administrada por el sistema operativo.

En el caso de un programa que se ejecuta en `user space`, el kernel detecta y maneja el fallo a partir de la excepción de hardware correspondiente, típicamente un `page fault`. Cuando la MMU detecta un acceso no permitido, transfiere el control al kernel, que analiza si la referencia puede resolverse legítimamente o si constituye un acceso inválido. Si el

acceso no es recuperable dentro del contexto del proceso, el kernel envía la señal `SIGSEGV` al programa responsable. El comportamiento por defecto del programa ante esa señal es finalizar su ejecución, y en muchos casos puede generarse además un `core dump` para depuración. No obstante, un programa también puede instalar un manejador para `SIGSEGV`; aun así, eso no convierte al error en algo seguro de continuar, sino que solo habilita tareas de registro, limpieza controlada o diagnóstico antes de terminar.

Si un error análogo ocurre en `kernel space`, por ejemplo dentro del propio kernel o de un módulo cargable, la situación es más grave. En ese contexto no se trata simplemente de terminar un proceso aislado, porque el código que falló se está ejecutando con privilegios máximos y comparte el espacio del núcleo. En consecuencia, el kernel suele producir un `oops` y registrar el fallo; según su severidad, el sistema puede continuar en un estado inestable o derivar en un `kernel panic`, comprometiendo al sistema completo.

2.8 8. ¿Se animan a intentar firmar un módulo de kernel? ¿Y documentar el proceso?

En esta etapa se intentó firmar un módulo de kernel propio y verificar su comportamiento con `Secure Boot` habilitado. La experiencia mostró primero que un módulo no firmado podía ser rechazado por la política de integridad del sistema, y luego permitió comprobar que, una vez firmado correctamente y asociado a una clave confiable mediante `MOK`, el mismo módulo podía cargarse y descargarse sin inconvenientes.

Como condición inicial, se verificó mediante los mensajes del kernel que la máquina estuviera ejecutándose con las restricciones de integridad asociadas a `Secure Boot`. Esta observación permitió contextualizar por qué un módulo externo no firmado sería rechazado al intentar insertarlo en el núcleo.

`dmesg | tail`

```
[ 0.015408] Using GB pages for direct mapping
[ 0.015657] secureboot: Secure boot enabled
[ 0.015657] RAMDISK: [mem 0x2e50d000-0x3304cfff]
```

2.1: Mensajes del kernel asociados al entorno con `Secure Boot` habilitado

Luego se compiló el módulo a partir del `Makefile` preparado para construir `signed_module.ko` usando el sistema de build del kernel en ejecución.

```
1  cd ../files_project/signed-kernel-module
2  make
```

```
Terminal
20:02 ~ doc/task4tp4 signed-kernel-module ls
20:02 ~ doc/task4tp4 signed-kernel-module make
make -C /lib/modules/6.17.0-29-generic/build M=/home/tomas/Documentos/facultad/siscom/computer_syst
ems_ramnt/TP4/files_project/signed-kernel-module modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-29-generic'
make[2]: se entra en el directorio '/home/tomas/Documentos/facultad/siscom/computer_systems_ramnt/T
P4/files_project/signed-kernel-module'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] signed_module.o
signed_module.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
   8 | int modulo_lin_init(void)
     | ^
signed_module.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
   17 | void modulo_lin_clean(void)
      ^
MODPOST Module.symvers
CC [M] signed_module.mod.o
CC [M] .module-common.o
LD [M] signed_module.ko
BTF [M] signed_module.ko
Skipping BTF generation for signed_module.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/tomas/Documentos/facultad/siscom/computer_systems_ramnt/TP4/
files_project/signed-kernel-module'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-29-generic'
20:02 ~ doc/task4tp4 signed-kernel-module
```

2.2: Compilación del módulo signed_module

Una vez obtenido el archivo .ko, se intentó cargarlo directamente para comprobar el efecto de Secure Boot sobre un módulo aún no firmado. En este punto, el kernel rechazó la operación, lo cual confirmó que la política de verificación de firmas estaba actuando sobre módulos externos.

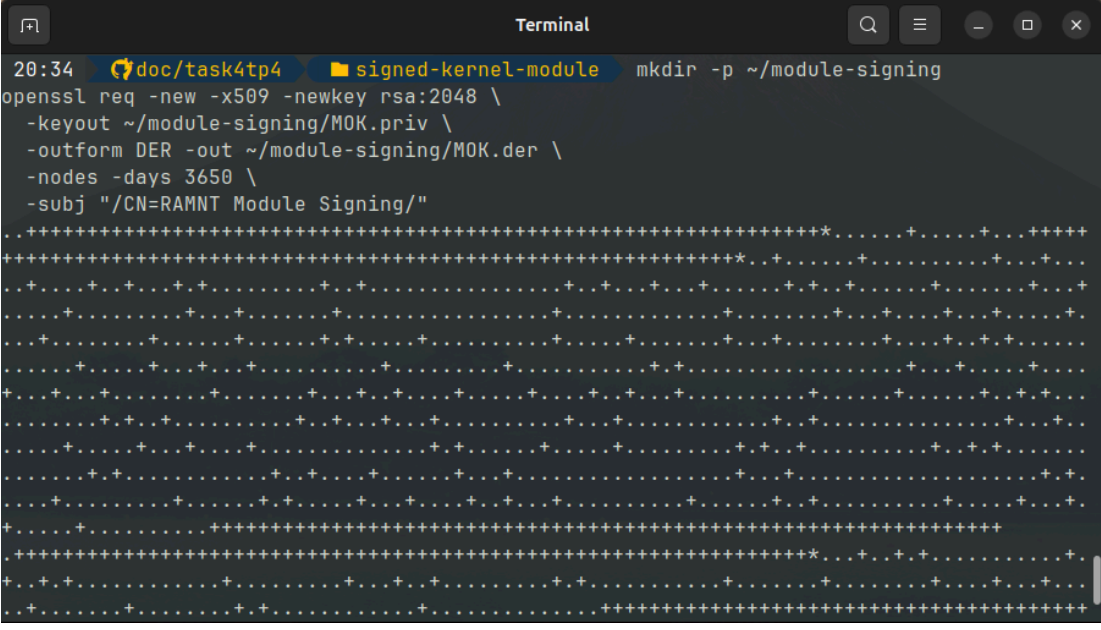
```
sudo insmod signed_module.ko
```

```
Terminal
20:30 ~ doc/task4tp4 signed-kernel-module sudo insmod signed_module.ko
insmod: ERROR: could not insert module signed_module.ko: Key was rejected by service
20:30 ~ doc/task4tp4 signed-kernel-module
```

2.3: Intento de carga del módulo sin firmar

Para resolverlo, se generó un par de claves: una clave privada para firmar el módulo y un certificado público en formato DER para ser enrolado posteriormente como clave confiable.

```
1  mkdir -p ~/module-signing
2  openssl req -new -x509 -newkey rsa:2048 \
3    -keyout ~/module-signing/MOK.priv \
4    -outform DER -out ~/module-signing/MOK.der \
5    -nodes -days 3650 \
6    -subj "/CN=RAMNT Module Signing/"
```

A screenshot of a terminal window titled "Terminal". The window shows a command prompt with the following text: `20:34 doc/task4tp4 signed-kernel-module mkdir -p ~/module-signing`. Below this, the command `openssl req -new -x509 -newkey rsa:2048 \` is entered, followed by several lines of options: `-keyout ~/module-signing/MOK.priv \`, `-outform DER -out ~/module-signing/MOK.der \`, `-nodes -days 3650 \`, and `-subj "/CN=RAMNT Module Signing/"`. The terminal output shows a series of asterisks and dots, indicating the progress of the key generation process.

2.4: Generación de la clave privada y del certificado público

Con la clave ya disponible, se procedió a firmar el módulo mediante el script `sign-file` provisto por los headers del kernel. Este paso incrusta la firma digital dentro del archivo `.ko`.

```
1  /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha512 \
2    ~/module-signing/MOK.priv \
3    ~/module-signing/MOK.der \
4    signed_module.ko
5  modinfo signed_module.ko
```



```
Terminal
20:48 doc/task4tp4 signed-kernel-module modinfo /home/tomas/Documentos/facultad/s
iscom/computer_systems_ramnt/TP4/files_project/signed-kernel-module/signed_module.ko
filename: /home/tomas/Documentos/facultad/siscom/computer_systems_ramnt/TP4/files_proje
ct/signed-kernel-module/signed_module.ko
author: GRUPO RAMNT
description: Modulo de kernel firmado
license: GPL
srcversion: E99B573793227188E1D319E
depends:
name: signed_module
retpoline: Y
vermagic: 6.17.0-29-generic SMP preempt mod_unload modversions
sig_id: PKCS#7
signer: RAMNT Module Signing
sig_key: 49:30:DF:A5:9E:FF:9D:14:72:61:C0:C8:06:27:79:BE:CD:E0:73:89
sig_hashalgo: sha512
signature: E0:74:8F:E9:4E:33:D7:BD:12:D5:98:62:8C:07:B2:D3:AF:93:D8:4A:
97:CC:D8:C4:8D:0D:F2:23:28:A2:7E:09:A3:DF:90:0F:41:3E:4D:20:
75:7A:F8:C0:E0:38:E6:B8:0E:05:B5:E8:B4:7C:F0:C4:F4:38:E3:85:
BA:4B:00:BD:9F:5F:11:65:B8:C7:3C:39:BF:60:DE:B3:DD:A7:68:02:
BE:FC:D4:0A:A5:6B:A3:C1:DD:67:1C:40:9D:56:C2:A8:F8:AB:5A:4A:
4E:A5:CA:86:82:18:99:0E:96:7F:C7:C8:66:17:D0:C6:DB:36:EB:D4:
23:7C:F1:3C:4F:85:60:4D:EF:8D:5F:8A:5D:34:56:26:4C:42:C6:BA:
CE:5B:98:AA:7D:1F:29:24:57:1C:87:7D:99:38:D3:10:2C:5D:75:83:
60:1D:2B:27:24:AD:33:42:6F:88:AA:F3:CF:93:C3:09:50:E9:6A:F8:
88:34:1D:9C:A9:0E:79:85:CB:4B:1E:3F:C3:93:CF:AA:08:4C:8A:45:
9C:39:F3:1B:BB:44:3E:24:28:FD:4C:E0:32:1D:17:23:0B:20:58:FC:
12:E2:33:21:E4:F6:FB:13:3C:3D:FB:B9:57:6A:8D:92:6F:EE:9B:3F:
F7:DD:60:66:FC:57:8B:52:56:7B:6E:55:3D:6F:6E:0A
20:48 doc/task4tp4 signed-kernel-module
```

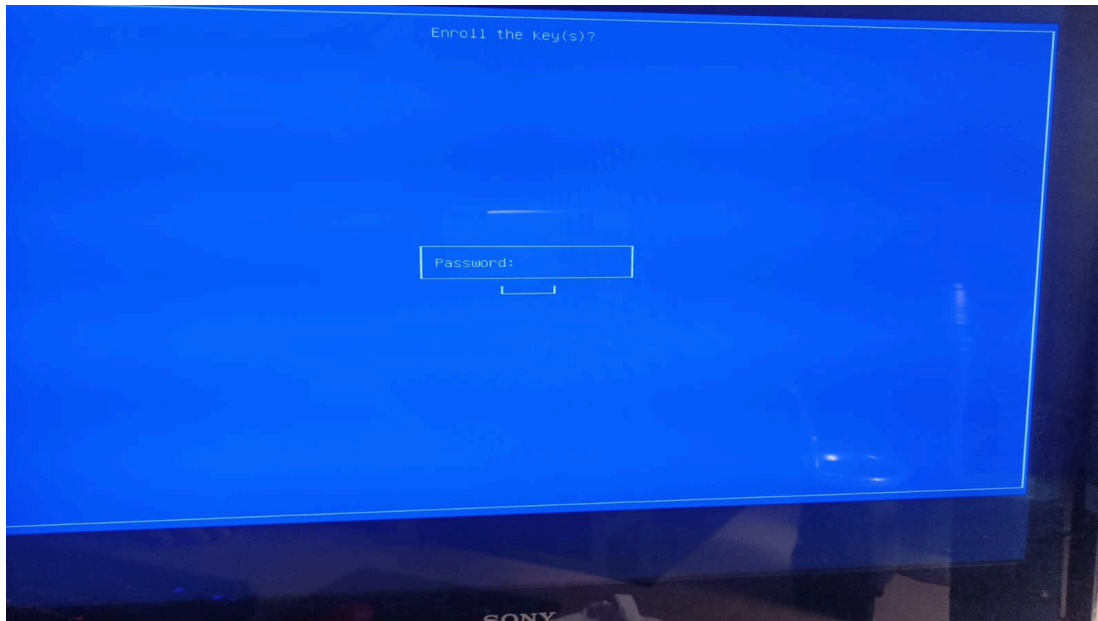
2.5: Firma del módulo y verificación con modinfo

Sin embargo, firmar el módulo no es suficiente por sí solo: el sistema también debe confiar en la clave utilizada. Por eso se importó el certificado público con `mokutil`, dejando pendiente su enrolamiento para el próximo arranque.

```
sudo mokutil --import ~/module-signing/MOK.der
```

```
20:48 doc/task4tp4 signed-kernel-module sudo mokutil --import ~/module-signing/MO
K.der
[sudo] contraseña para tomas:
input password:
input password again:
20:52 doc/task4tp4 signed-kernel-module
```

2.6: Creación de la clave pública enrolada en MOK



2.7: Confirmación de la clave en MOK Manager

Tras reiniciar el equipo, apareció la interfaz de MOK Manager, desde la cual se confirmó manualmente la incorporación de la clave. Este paso resulta necesario porque la incorporación a la base de confianza no se realiza de manera automática desde el sistema operativo en ejecución, sino durante el proceso de arranque seguro.

Una vez completado el enrolamiento, se verificó desde Linux que la clave pública hubiera quedado efectivamente registrada entre las claves aceptadas por el sistema.

```
mokutil --list-enrolled
```

```

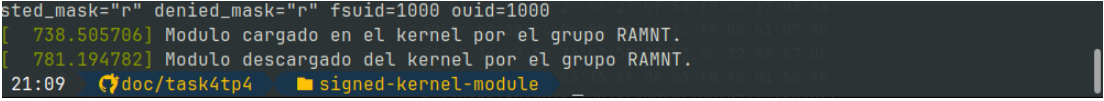
Terminal
sudo: 1 intento de contraseña incorrecto
21:04 doc/task4tp4 signed-kernel-module
21:04 doc/task4tp4 signed-kernel-module modinfo signed_module.ko
Filename: /home/tomas/Documentos/facultad/siscom/computer_systems_ramnt/
TP4/files_project/signed-kernel-module/signed_module.ko
Author: GRUPO RAMNT
Description: Modulo de kernel firmado
License: GPL
srcversion: E99B573793227188E1D319E
Depends:
name: signed_module
prepoline: Y
vermagic: 6.17.0-29-generic SMP preempt mod_unload modversions
sig_id: PKCS#7
signer: RAMNT Module Signing
sig_key: 49:30:DF:A5:9E:FF:9D:14:72:61:C0:C8:06:27:79:BE:CD:E0:73:89
sig_hashalgo: sha512
signature: E0:74:8F:E9:4E:33:07:80:12:D5:98:62:8C:07:B2:D3:AF:93:D8:4A:
97:CC:08:04:80:0D:F2:23:28:A2:7E:09:A3:DF:90:0F:41:3E:4D:20:
75:7A:F8:00:E0:38:E6:B8:0E:05:B5:E8:B4:7C:F0:C4:F4:38:E3:85:
BA:48:00:BD:9F:5F:11:65:B8:C7:3C:39:BF:60:DE:B3:D0:A7:68:02:
BE:FC:D4:0A:A5:68:A3:C1:00:67:1C:40:90:56:C2:A8:F8:AB:5A:4A:
4E:A5:CA:86:82:18:99:0E:96:7F:C7:C0:66:17:D0:C6:DB:36:E8:D4:
23:7C:F1:3C:4F:05:60:4D:EF:8D:5F:8A:50:34:56:26:4C:42:C6:8A:
CE:58:98:AA:70:1F:29:24:57:1C:87:7D:99:38:D3:10:2C:5D:75:83:
60:10:2B:27:24:AD:33:42:6F:88:AA:F3:CF:93:C3:09:50:E9:6A:F8:
88:34:1D:9C:A9:06:79:85:C8:4B:1E:3F:C3:93:CF:AA:08:4C:8A:45:
9C:39:F3:1B:B8:44:3E:24:28:FD:4C:E0:32:1D:17:23:0B:20:58:FC:
12:E2:33:21:E4:F6:FB:13:3C:3D:FB:B9:57:6A:8D:92:6F:EE:9B:3F:
F7:D0:68:66:FC:57:8B:52:56:78:6E:55:30:6F:6E:8A
21:04 doc/task4tp4 signed-kernel-module
0: 0:[tmux]
e0:68:02:8f:fb:f9:47:1d:7d:a2:01:c6:07:51:c4:9a:c21:04:59 [40/106]
dd:cf:a3:5d:ed:92:bb:be:d1:fd:e6:ec:1f:33:51:73:04:be:
3c:72:b0:7d:88:f8:01:ff:98:7d:cb:9c:e0:69:39:77:25:47:
71:88:b1:8d:27:a5:2e:a8:f7:3f:5f:80:69:97:3e:a9:f4:99:
14:db:ce:03:0e:0b:66:c4:1c:6d:bd:b8:27:77:c1:42:94:bd:
fc:6a:0a:bc
[key 2]
SHA1 Fingerprint: b1:43:52:10:e6:69:83:bc:6a:31:b6:ab:b0:9a:37:d2:cc:04:d4
:ff
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
49:30:df:a5:9e:ff:9d:14:72:61:c0:c8:06:27:79:be:cd:e0:73:89
Signature Algorithm: sha256WithRSAEncryption
Issuer: CN=RAMNT Module Signing
Validity
Not Before: May 23 23:48:11 2026 GMT
Not After : May 20 23:48:11 2036 GMT
Subject: CN=RAMNT Module Signing
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:ee:78:f3:e5:aa:d0:bf:c2:4f:e2:ae:b9:8f:2c:
90:cd:55:91:5f:47:d3:37:91:50:52:76:0e:ef:6b:
b0:fd:80:6d:b0:6b:fb:8f:ba:ab:65:1e:c9:fa:60:
1f:b3:c9:b7:e9:2c:66:23:bf:54:f7:66:17:d3:aa:
a2:fc:b6:58:c4:0c:00:db:ab:fc:f8:b0:b1:07:9b:
33:39:14:b3:5a:32:b2:52:03:a3:6b:22:80:e7:dc:
"tomas-H410M-N-V3" 21:06 23-may-24

```

2.8: Comprobación de la clave del modulo con la clave registrada en linux

Con la clave ya confiable, fue posible cargar y descargar el módulo firmado correctamente. Además, el propio módulo dejó evidencia en los registros del kernel mediante `printk`, imprimiendo mensajes al cargar y al descargar.

```
1  sudo insmod signed_module.ko
2  dmesg | tail
3  sudo rmmod signed_module
4  dmesg | tail
```



```
sted_mask="r" denied_mask="r" fsuid=1000 ouid=1000
[ 738.505706] Modulo cargado en el kernel por el grupo RAMNT.
[ 781.194782] Modulo descargado del kernel por el grupo RAMNT.
21:09 doc/task4tp4 signed-kernel-module
```

2.9: Mensajes de carga y descarga del módulo en los registros del kernel

En conclusión, la práctica permitió comprobar que la aceptación de un módulo con **Secure Boot** habilitado no depende solamente de compilarlo correctamente, sino de dos condiciones simultáneas: que el archivo esté firmado y que la clave correspondiente forme parte de la cadena de confianza del sistema. Cuando ambas condiciones se cumplieron, el módulo pudo cargarse y descargarse normalmente.

2.9 10. ¿Qué pasa si mi compañero con **Secure Boot** habilitado intenta cargar un módulo firmado por mí?

Si el módulo está firmado con una clave personal que no forma parte de la cadena de confianza del sistema de destino, la carga será rechazada en aquellos entornos donde **Secure Boot** implique verificación estricta de firmas de módulos. En ese escenario, no alcanza con que el módulo esté firmado: la clave pública correspondiente debe encontrarse registrada en un anillo de claves confiable para el kernel, por ejemplo mediante **MOK** o a través de claves incorporadas al propio sistema.

En consecuencia, si un compañero intenta cargar un módulo firmado por otra persona pero cuya clave no fue previamente enrolada en su equipo, lo esperable es que el kernel deniegue la operación por firma no confiable. Solo en el caso de que esa clave haya sido incorporada explícitamente al conjunto de claves aceptadas podrá verificarse la firma y permitirse la carga del módulo.

2.10 11. Dada la siguiente nota

<https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>

2.10.1 a. ¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

Según la nota, la consecuencia principal fue que múltiples equipos configurados con arranque dual entre Windows y Linux dejaron de poder iniciar Linux cuando `Secure Boot` permanecía habilitado. El parche, publicado por Microsoft para mitigar una vulnerabilidad de GRUB, terminó provocando fallas de validación asociadas a `shim` y `SBAT`, lo que derivó en errores de política de seguridad durante el arranque.

2.10.2 b. ¿Qué implicancia tiene desactivar `Secure Boot` como solución al problema descrito en el artículo?

La nota indica que desactivar `Secure Boot` es una alternativa para recuperar el arranque, pero también subraya que dicha decisión puede no ser aceptable según las necesidades de seguridad del usuario. El artículo incluso plantea que una mejor solución transitoria es eliminar la política `SBAT` aplicada por Microsoft, ya que ello conserva al menos parte de los beneficios de `Secure Boot`, aun cuando el sistema continúe expuesto a ataques que exploten CVE-2022-2601.

2.10.3 c. ¿Cuál es el propósito principal del `Secure Boot` en el proceso de arranque de un sistema?

De acuerdo con la bibliografía indicada, el propósito principal de `Secure Boot` es impedir que, durante el arranque, el sistema cargue firmware o software malicioso no confiable. Es, por tanto, un mecanismo de integridad orientado a proteger la cadena de arranque antes de que el sistema operativo complete su inicialización.

3 Respuestas

3.1 Desafío 1

3.1.1 ¿Qué es `checkinstall` y para qué sirve?

`checkinstall` es una herramienta orientada a sistemas GNU/Linux y otros entornos tipo Unix cuya finalidad es registrar una instalación realizada desde código fuente y transformarla en un paquete administrable por el sistema de paquetes de la distribución, típicamente `deb`, `rpm` o `tgz`. En lugar de ejecutar un `make install` convencional sin trazabilidad, `checkinstall` monitorea los archivos creados o modificados durante la instalación y construye, a partir de esa información, un paquete binario equivalente.

Su utilidad principal radica en que permite mantener control administrativo sobre software compilado manualmente. Esto facilita la posterior desinstalación, la auditoría de archivos instalados, la replicación de la instalación en otras máquinas compatibles y una mejor integración con las políticas de mantenimiento del sistema. En términos prácticos, constituye una solución intermedia entre la instalación informal desde fuentes y el empaquetado completo mantenido por la distribución.

3.2 Desafío 2

3.2.1 ¿Qué funciones tiene disponibles un programa y qué funciones tiene disponibles un módulo?

Un programa de espacio de usuario dispone, en primer término, de las funciones provistas por las bibliotecas de usuario, como la biblioteca estándar de C, y accede a servicios privilegiados por medio de llamadas al sistema. Por ello, operaciones como abrir archivos, reservar memoria dinámica, crear procesos o establecer comunicaciones de red se realizan a través de interfaces de usuario que luego son traducidas a `syscalls` hacia el kernel.

Un módulo del kernel, en cambio, no utiliza la biblioteca estándar como entorno de ejecución principal, sino que se integra directamente con la API interna del kernel. Sus funciones disponibles son aquellas que el núcleo exporta explícitamente mediante símbolos y subsistemas internos, tales como `printk`, `kmalloc`, `kfree`, `copy_to_user`, `copy_from_user`,

`request_irq`, o las rutinas de registro de dispositivos y controladores. En consecuencia, un módulo posee acceso a capacidades de mucho mayor privilegio, pero también opera bajo restricciones más severas: un error puede comprometer la estabilidad completa del sistema.

3.2.2 Espacio de usuario y espacio del kernel

El sistema operativo separa la ejecución en dos dominios fundamentales. El espacio de usuario es el ámbito donde corren las aplicaciones ordinarias, con privilegios limitados y aislamiento entre procesos. Allí, cada programa dispone de su propio espacio de direcciones virtuales y no puede acceder directamente ni al hardware ni a la memoria del kernel.

El espacio del kernel, por el contrario, es el dominio privilegiado desde el cual el sistema operativo administra memoria, interrupciones, planificación, dispositivos y políticas de protección. Los módulos cargables se ejecutan en este espacio y comparten el mismo contexto privilegiado del núcleo. La separación entre ambos dominios es esencial para la seguridad y la robustez del sistema, ya que impide que una falla en una aplicación común afecte de manera directa a la totalidad del sistema operativo.

3.2.3 Espacio de datos

En un programa de usuario, el espacio de datos forma parte de su espacio de memoria virtual e incluye, de manera general, datos globales inicializados, datos globales no inicializados, heap y stack. Este espacio pertenece al proceso y queda protegido por mecanismos de aislamiento; si el programa finaliza, su memoria es recuperada por el sistema.

En un módulo del kernel, en cambio, el espacio de datos no está aislado como el de un proceso ordinario, sino que queda incorporado al espacio global del kernel mientras el módulo permanezca cargado. Por ello, las estructuras de datos de un módulo deben diseñarse con especial cuidado respecto de concurrencia, sincronización y tiempo de vida. Una escritura inválida en ese contexto no compromete solamente al módulo, sino potencialmente al núcleo completo.

3.2.4 Drivers e investigación del contenido de `/dev`

Un driver es un componente de software, frecuentemente implementado como módulo del kernel, cuya responsabilidad es mediar entre el sistema operativo y un dispositivo físico o lógico. Su función consiste en traducir operaciones genéricas del sistema en acciones

concretas sobre el hardware o sobre una interfaz virtual. En Linux, muchos drivers se presentan al resto del sistema mediante archivos especiales de dispositivo.

El directorio `/dev` contiene precisamente esos archivos especiales, conocidos como nodos de dispositivo. No almacenan datos convencionales como un archivo común; su propósito es ofrecer una interfaz uniforme para interactuar con dispositivos y servicios del sistema. Dentro de `/dev` es habitual distinguir dos grandes clases:

- Dispositivos de carácter, que transfieren información como flujo secuencial de bytes, por ejemplo terminales, puertos serie o `/dev/null`.
- Dispositivos de bloque, que operan sobre bloques direccionables, como discos y particiones.

Cada entrada de `/dev` se asocia internamente con un número mayor (`major`) y un número menor (`minor`). El número mayor identifica el driver responsable de atender las operaciones sobre ese archivo de dispositivo, mientras que el menor permite distinguir instancias o subdispositivos administrados por ese mismo driver. En sistemas modernos, muchas de estas entradas son creadas dinámicamente por `udev`, en respuesta a los eventos de detección de hardware generados por el kernel.

Desde una perspectiva conceptual, `/dev` constituye el punto de encuentro entre el espacio de usuario y los controladores del kernel: una aplicación abre, lee, escribe o configura un nodo de dispositivo, y el kernel redirige dichas operaciones al driver correspondiente. Este esquema materializa uno de los principios centrales de Unix y Linux: tratar múltiples recursos del sistema mediante una interfaz homogénea basada en archivos.

3.3 Bibliografía

- Red Hat, *Restrictions Imposed by UEFI Secure Boot*:
<https://access.redhat.com/articles/1351013>
- Red Hat, *Managing kernel modules / signing kernel modules for secure boot*:
https://access.redhat.com/documentation/es-es/red_hat_enterprise_linux/8/html/managing_monitoring_and_updating_the_kernel/signing-kernel-modules-for-secure-boot_managing-kernel-modules
- The Linux Kernel Module Programming Guide:
<https://sysprog21.github.io/lkmpg/#what-is-a-kernel-module>
- Opensource.com, *Dynamic tracing in Linux user and kernel space*:
<https://opensource.com/article/17/7/dynamic-tracing-linux-user-and-kernel-space>

- Ars Technica, “*Something has gone seriously wrong,*” *dual-boot systems warn after Microsoft update*: <https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>